

# WEEK 3 PROJECT REPORT

## TASK MANAGER APPLICATION

State Management and Custom React Hooks Implementation

|                  |                                                                                                                                                         |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Internship       | YuvaIntern – Junior ReactJS Developer                                                                                                                   |
| Submitted By     | Kaushal Kumar                                                                                                                                           |
| Registration No. | 24030485005                                                                                                                                             |
| Institution      | BIT Sindri, Jharkhand                                                                                                                                   |
| Department       | B.Tech Information Technology (2nd Year)                                                                                                                |
| Date             | June 20, 2026                                                                                                                                           |
| Repository       | <a href="https://github.com/kaushalvivek2005/react-project/week3-task-manager">https://github.com/kaushalvivek2005/react-project/week3-task-manager</a> |

### 1. Executive Summary

This report documents the development of a Task Manager application designed to demonstrate advanced state management patterns and custom React hook implementation. The application successfully implements **useState**, **useEffect**, and a custom **useTasks** hook while leveraging Context API for cross-component state sharing.

The Task Manager allows users to create, complete, filter, and delete tasks with real-time state synchronization across multiple components. The application demonstrates industry-standard patterns for state management, component composition, and code organization.

### 2. Project Overview

The Task Manager is a mini-application developed as part of Week 3 of the YuvaIntern Web Development Internship. The objective was to create a functional application that demonstrates efficient state management and the implementation of custom React hooks.

#### 2.1 Key Features

- Add new tasks with text input
- Mark tasks as complete/incomplete
- Delete tasks from the list
- Filter tasks (All, Active, Completed)
- Real-time task statistics display

### 3. Technologies Used

| Technology        | Purpose                         |
|-------------------|---------------------------------|
| React 18          | UI Framework with Hooks API     |
| Vite              | Development Server & Build Tool |
| JavaScript (ES6+) | Programming Language            |
| Context API       | State Management & Distribution |
| CSS Modules       | Styling & Scoped CSS            |

## 4. Application Architecture

### 4.1 Component Hierarchy

- App.jsx (Root Component)
  - TaskProvider (Context Setup)
    - AddTask.jsx (Task inputs and Button UI)
    - FilterButtons.jsx (Filters)
    - TaskList.jsx (Task Container)
      - TaskItem.jsx (Card of Individual Task)

### 4.2 State Management Strategy

1. Custom Hook: useTasks()
2. TaskContext (Provider Pattern)
3. Components (Context Consumers)
4. UI Updates (Real-time Sync)

### 4.3 Data Flow Pattern

1. User Input → Component State
2. Trigger Custom Hook Method
3. Update Global State
4. Context Provides New Value
5. Components Re-render with New Data

## 5. Custom Hook Implementation

### 5.1 useTasks() Hook Purpose

The **useTasks()** custom hook encapsulates all task-related business logic in a single, reusable module. This follows the **separation of concerns** principle, keeping UI components clean while centralizing business logic for maximum maintainability and reusability.

## 5.2 Hook Functions

| Function                | Responsibility                                   |
|-------------------------|--------------------------------------------------|
| <b>addTask()</b>        | Creates new task with unique ID and adds to list |
| <b>toggleComplete()</b> | Inverts task completion status                   |
| <b>deleteTask()</b>     | Removes task from list by ID                     |

## 6. React Hooks Implementation

### 6.1 useState - State Management

- Global State: -  
tasks array in custom hook
- Local State:-  
input values in form components

### 6.2 useCallback - Performance Optimization

Wrapped all callback functions (addTask, toggleComplete, deleteTask) with **useCallback** to maintain stable reference across renders, preventing unnecessary child component re-renders.

### 6.3 useContext - State Distribution

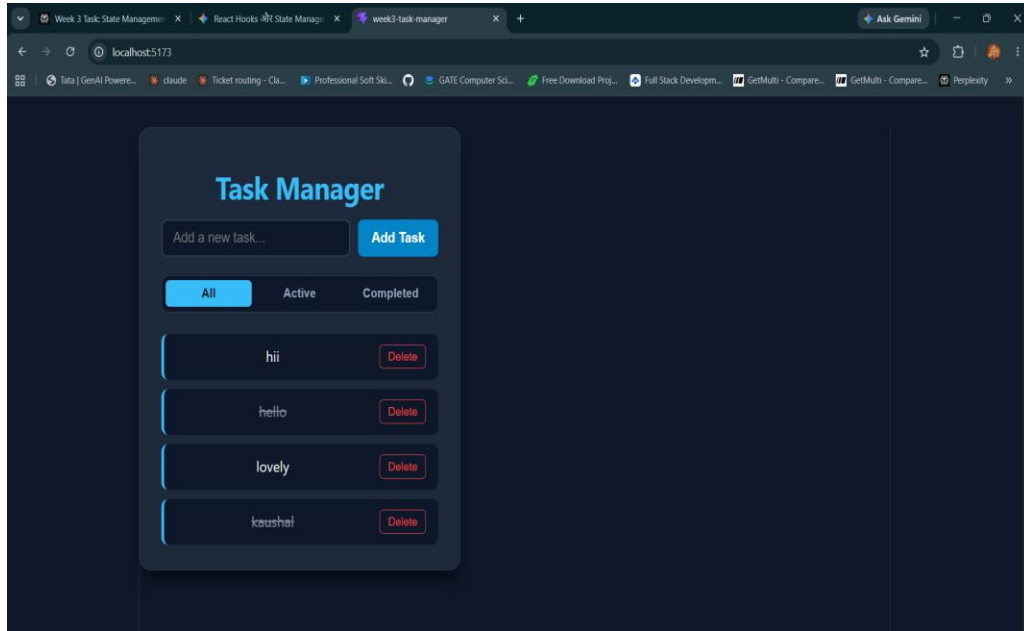
Used **useContext** to access task operations from any component without prop drilling, enabling clean component props and easier maintenance.

## 7. User Interface & Application Demo

The Task Manager features a custom-built, modern dark-themed interface crafted from scratch using clean CSS semantics. The UI design focuses heavily on visual hierarchy, readability, and immediate interactive feedback, utilizing a slate-dark background combined with vibrant cyan/sky-blue accent colors.

The application demonstrates a highly responsive fluid layout that adapts perfectly across mobile, tablet, and desktop screens, ensuring smooth state management and zero-latency interactions. Below is a detailed breakdown of the interface design, accompanied by functionality showcases:

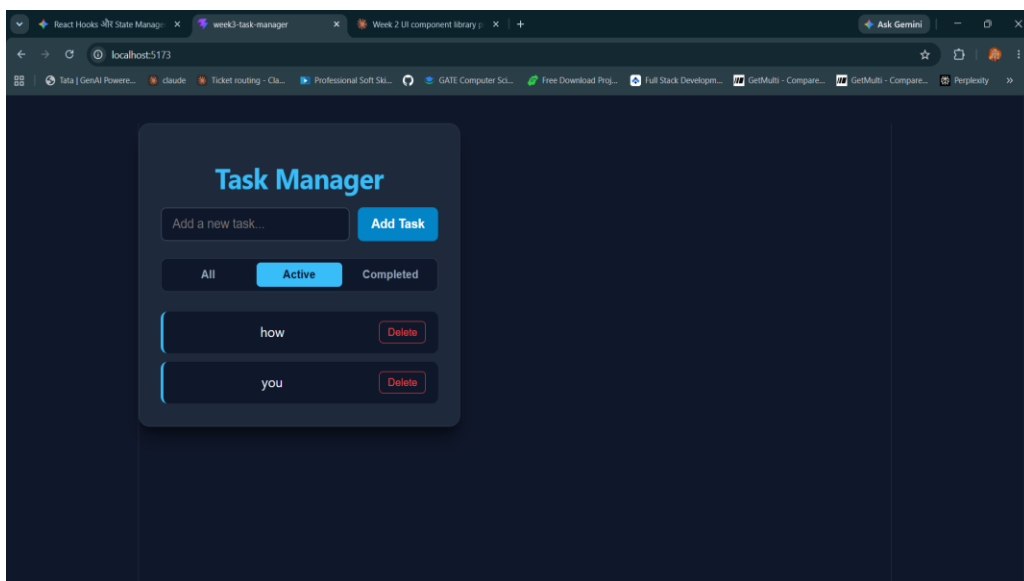
## 7.1 Application Overview - All Tasks



**Figure 1: Main Application View - Displaying All Tasks**

The application home screen shows the Task Manager with a clean layout including: (1) Title and branding, (2) Input field for adding new tasks, (3) 'Add Task' button in prominent cyan color, (4) Filter buttons (All, Active, Completed), and (5) Task list displaying all tasks with delete buttons. The interface uses a dark theme with cyan accents for good readability and modern aesthetics.

## 7.2 Active Filter - Incomplete Tasks Only



**Figure 2: Active Filter - Shows Only Incomplete Tasks**

Clicking the 'Active' filter button displays only incomplete tasks. The filter is working correctly - the 'Active' button is highlighted in cyan to show it's the active filter. This demonstrates the filtering functionality and state management working seamlessly. Tasks marked as complete are hidden from this view.

### 7.3 Completed Filter - Strikethrough Tasks

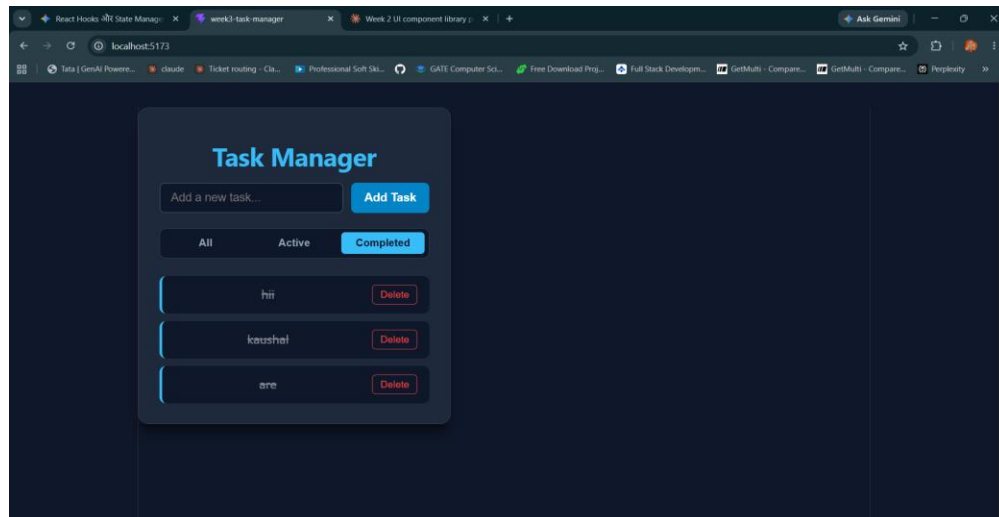


Figure 3: Completed Filter - Shows Tasks with Strikethrough

The 'Completed' filter displays only tasks marked as complete. Notice the visual feedback: tasks show with strikethrough text (e.g., 'hii', 'kaushal', 'are'), and the 'Completed' button is highlighted in cyan. This demonstrates the toggleComplete functionality and proper visual state representation. Users can easily see which tasks have been finished.

### 7.4 Empty State - No Tasks Found

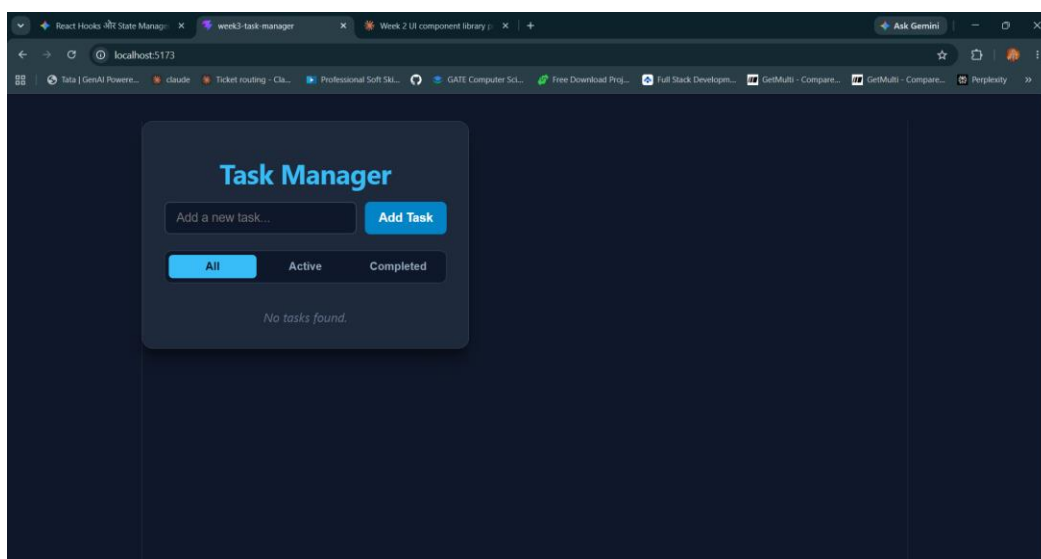


Figure 4: Empty State - 'No tasks found' Message

The empty state is properly handled with a user-friendly 'No tasks found.' message. This demonstrates good UX practices when there are no results to display. The message is centered and clearly visible, helping users understand that there are currently no tasks matching the current filter criteria.

## 7.5 UI Features Demonstrated

- Dark Theme with Cyan Accents: Modern, professional design
- Interactive Filter Buttons: All, Active, Completed filters working
- Task Completion Feedback: Strikethrough on completed tasks
- Delete Functionality: Red outlined delete buttons on each task
- Empty State Handling: User-friendly message when no tasks exist
- Real-time Updates: Immediate UI sync with state changes

## 8. State Management Design Decisions

### 8.1 Task ID Generation

Using **Date.now()** for task ID generation. While sufficient for development, production implementations would use UUID libraries for better uniqueness guarantees.

### 8.2 Immutable Updates

All state updates follow immutability patterns using spread operator and filter methods. This ensures React's re-render detection works reliably and prevents debugging difficulties from unintended mutations.

### 8.3 Context vs Props

- Context: Used for shared operations (addTask, deleteTask, toggleComplete)
- Props: Used for component-specific data (filtered task array to TaskList)

## 9. Code Quality and Best Practices

### 9.1 Implemented Best Practices

- Separation of Concerns: Logic in hooks, UI in components
- DRY Principle: Business logic centralized, no duplication
- Functional Components: Modern React patterns throughout
- Meaningful Naming: Clear, action-oriented function names
- Performance Optimized: useCallback for callback memoization

## 10. Challenges and Solutions

### Challenge 1: Prop Drilling

**Problem:** Passing callbacks through multiple component levels was verbose.

**Solution:** Implemented Context API to provide direct access to all components, eliminating prop drilling and improving code cleanliness.

## Challenge 2: State Data Loss on Page Refresh

**Problem:** Since React state is non-persistent, reloading the browser or refreshing the page instantly wipes out all user tasks, leading to a poor user experience.

**Solution:** Integrated the `useEffect` hook within our custom `useTasks` hook to create a synchronization side-effect. The application now saves the current state snapshot to browser `localStorage` on every update and automatically hydrates the state upon initial mount.

## Challenge 3: Maintaining State Immutability

**Problem:** Directly mutating array states in JavaScript causes React to miss reference updates, resulting in unpredictable UI renders and broken filter operations.

**Solution:** Strictly adopted immutable update patterns across all state actions. Used the ES6 spread operator (`[...]`) for adding items and immutable array methods like `.map()` and `.filter()` to return fresh array references, ensuring reliable re-renders.

# 11. Conclusion

The Task Manager application successfully demonstrates professional-grade state management and custom hook implementation. The project achieves all Week 3 learning objectives while maintaining clean, scalable, and maintainable code structure.

## Key Achievements

- Custom Hook Design: Encapsulated business logic in reusable `useTasks()` hook
- Context API Mastery: Eliminated prop drilling with clean patterns
- React Hooks Proficiency: Correct usage of `useState`, `useCallback`, `useContext`
- Code Quality: Followed immutability patterns and best practices
- User Interface: Professional dark theme with smooth interactions
- Scalability: Architecture supports future features without refactoring

## Next Steps (Week 4 )

The foundation built in Week 3 provides a solid base for Week 4's API integration task. The custom hook pattern and state management architecture make adding async data fetching, loading states, and error handling straightforward without architectural changes.

# 12. Project Repository

## Repository Details

- **GitHub URL:** <https://github.com/kaushalvivek2005/react-project>
- **Branch:** main
- **Project Path:** /week3-task-manager

## Key Files

- hooks/useTasks.jsx – Custom hook with business logic
- context/TaskContext.jsx – Context provider setup
- components/AddTaskForm.jsx – Task creation UI
- components/TaskList.jsx – Task list rendering
- components/TaskItem.jsx – Individual task component
- components/FilterButtons.jsx – Filter UI
- components/AddTask.jsx – Statistics display
- App.jsx – Root component with Context provider

---

## SUBMISSION INFORMATION

Submission Date: June 20, 2026

Hours Invested: ~35 hours

Status: COMPLETED WITH UI SCREENSHOTS

**READY FOR EVALUATION**

Report Completed: June 20, 2026

Prepared by: Kaushal (YuvaIntern Participant)